

SD Basic - Status, Encryption and the Machine

Status codes, checksums, encryption, the file mask, the @variables, os.execute and the error log.

This page continues SD Basic - System and Environment.

STATUS() and OS.ERROR()

`status()` carries the result of the last operation that sets one. It is overwritten constantly, so **read it into a variable on the line after the call you care about**, not three lines later. Every `status()` value quoted in this document set was captured that way.

`os.error()` carries the operating system's own error number from the last call that made one, and is `0` when nothing has failed.

Codes worth knowing:

1004	item not found
1006	bad action key
1008	action failed — <code>os.error()</code> has the detail
1011	timeout
3001	subfile not found
3006	record not found
3007	no VOC record — from a failed <code>open</code> of a name that is not in the VOC
3021	cannot gain exclusive access to a file
3023	write or delete with no lock held
7005 / 7012 / 7013	socket: cannot connect / cannot bind / closed

CHECKSUM()

```
checksum(string)
```

<code>checksum('ABC')</code>	<code>451</code>
<code>checksum('abc')</code>	<code>291</code>
<code>checksum('ACB')</code>	<code>448</code>
<code>checksum('')</code>	<code>0</code>
<code>checksum('A' : @fm : 'B')</code>	<code>-325</code>

It is case sensitive and order sensitive, and **it can be negative**, so a program that stores it needs a signed field. It is a change detector, not a digest: it is short, and it is not a security primitive.

SDENCRYPT() and SDDECRYPT()

```
sdencrypt(data, key, encoding)
sddecrypt(data, key, encoding)
```

Three arguments. The encoding is `201` for hex or `202` for base64.

A passphrase is not a key, and an ordinary program cannot make one. `sdencrypt('The quick brown fox', 'secretkey', 202)` returns **nothing** and sets `status()` to **10204**, a key length error. The key has to be an encoded 256-bit key, and the function that derives one from a password is `sdest()`, which is internal-only. **From an ordinary account these two functions have no usable key**, and there is no way in.

UMASK()

```
umask(n)
```

Sets the file creation mask and returns the **previous** value. A negative argument asks without setting:

CALL	RETURNED	MASK AFTERWARDS
<code>umask(-1)</code>	<code>2</code>	<code>2</code> — unchanged
<code>umask(18)</code>	<code>2</code> — the old value	<code>18</code>
<code>umask(-1)</code>	<code>18</code>	<code>18</code>
<code>umask(2)</code>	<code>18</code>	<code>2</code> — restored

So the way to change it temporarily is to keep what the setting call returned and pass it back.

SENTENCE() and the @variables

`sentence()` and `@sentence` are the same thing. Both hold the command line that started the program: `RUN BP ZZMATH`.

	VALUE
<code>@who</code>	<code>DON</code> — upper case
<code>@logname</code> , <code>@user</code>	<code>don</code> — lower case
<code>@path</code>	<code>/cygdrive/c/ProgramData/SD/user_accounts/don</code>
<code>@sdsys</code>	<code>C:\ProgramData\SD\sdsys</code>
<code>@user.no</code>	<code>67</code> , the same as <code>system(18)</code>
<code>@tty</code>	empty in a piped session
<code>@system.return.code</code>	<code>1</code>
<code>@user.return.code</code>	<code>0</code>
<code>@crtwide</code> / <code>@crthigh</code>	<code>200</code> / <code>9999</code> — whatever <code>TERM</code> last set

`@who` and `@logname` **differ in case for the same account**. Compare them with `upcase()` on both sides or the test fails on a machine where it worked.

`set.exit.status` sets what SD returns to the operating system when the session ends.

OS.EXECUTE

```
os.execute command {capturing variable}
```

It is gated per account, and a refusal aborts the program rather than setting a status. In an ordinary account:

```
don is not permitted to use OS.EXECUTE at line 10 of .../BP.OUT/ZZMATH
```

The program stops there. There is no `else`, no `on error` and no status to test, so **a program that may run in an account without the right must not reach the statement at all.**

Permission is field 2 of the account's record in the system `os.users` file — not a VOC entry and not a Windows privilege. An administrator's session passes regardless. Ask your administrator to grant it; there is nothing a program can do about it.

LOGMSG

```
logmsg text
```

Writes a line to SD's error log. It has no return value, and **what** `status()` **says afterwards is whatever the previous statement left there** — worth knowing because it looks like a result.

What is not here

A whole family of functions is internal-only, and the compiler's complaint names something else entirely. This program, in an ordinary account:

```
v1 = kernel(28, 0)
v2 = ospath('C:', 1)
v5 = option('X')
v6 = pterm(1, '')
```

compiles to, at the **last line of the program**:

```
41: Matrix KERNEL is not referenced in a DIM statement
41: Matrix OPTION is not referenced in a DIM statement
```

```
41: Matrix OSPATH is not referenced in a DIM statement
41: Matrix PTERM is not referenced in a DIM statement
WARNING: KERNEL is not assigned a value
```

A name the compiler does not know as a function is read as a **matrix reference**, so the error is about a `dim` statement you never wrote, at a line number nowhere near the call. **If a function you are sure exists produces that, it exists and this account may not call it.** A call with three arguments gives a different but equally misleading answer — `sdx(101, 'pw', 'salt')` is "*Right bracket not found where expected*", because a matrix takes at most two subscripts.

Internal-only: `kernel()` — and therefore the Windows path conversion — `ospath()`, `option()`, `pterm()`, `sdx()`, `testlock()` and `getlocks()`. The compiler's list is longer than that; those seven are the ones this page put in front of it. They are reachable only from a program compiled with `$internal`, which additionally requires an administrator in the `SDSYS` account.

And some statements are restricted the same way. These are "*Unrecognised statement*" in an ordinary account: `set.modes`, `reset.modes`, `remove.token`, `release.lock`, `como`, `quit`, `keyboard.input`, `writepkt`, and the whole debugging family — `debug.on`, `debug.off`, `debug.set`, `breakpoint` and `watch`.

`errmsg` **is in the compiler's statement table and does not exist.** It compiles to "*Unrecognised statement*" for everybody. Its opcode was removed in July 2024 and the name was left behind in the table. **Being in the table is not evidence a statement exists.**

`sendmail`, `chgphant()` and `ccall()` **compile for an ordinary account**, and each needs something outside SD before it can do anything: `sendmail` a mail relay, `chgphant` a phantom to change, and `ccall` a C function registered into the executable. Nothing on this page depends on them.

`procread` and `procwrite` belong to the PROC language rather than to BASIC and are only meaningful inside one.

See also

SD Basic - Program Control · SD Basic - Data Conversion · SD Basic - Debugging · SD Basic - Locks and Transactions.